```
Car::whatAmI();

Output:
I am a Vehicle
```

Oops. Even though we're calling `whatAmI()` in the context of the `Car` class, the method is using the context of the class where it is declared: `self` will always resolve to the declaring class. What we need is a way to resolve `$iAm` to the called class (in this case, `Car`). Fortunately, PHP has a feature called *late static binding*, which provides exactly what we need. If we swap `self` for `static` in `whatAmI()`:

```
    public static function whatAmI(): void
    {
        print "I am a " . static::$iAm . PHP_EOL;
    }

Output:
I am a Car
```

Readonly

PHP 8.1 introduces a new keyword, `readonly`, which is used to indicate that class properties can only be initialized once. Once set, the value of a `readonly` property cannot be overwritten, not even by the class itself. PHP 8.2 also allows entire classes to be declared `readonly`, which has the effect of setting `readonly` on all class properties.

Some caveats with `readonly` properties: they cannot be initialized outside of the class, so they must be assigned a value either in a constructor or setter method. Using a setter is not preferred because they can be called multiple times, as the following example shows:

```
class DataObj
{
    public readonly string $nonConstructedProp;

    public function __construct(
        public readonly string $someProp,
    ) {
    }
```